

Assignment 2

Hadeer Ghoneim

Verification Diploma

Question 1

```
1  module test_dynamic_array();
2
3      int dyn_arr2[] = '{9, 1, 8, 3, 4, 4}';
4      int dyn_arr1[];
5
6      initial begin
7
8          dyn_arr1 = new[6];
9
10         foreach (dyn_arr1[i]) begin
11             dyn_arr1[i] = i;
12         end
13
14         $display("dyn_arr1 = %p", "size = %0d", dyn_arr1, dyn_arr1.size());
15
16         dyn_arr1.delete();
17
18         dyn_arr2.reverse;
19         $display("dyn_arr2 = %p", dyn_arr2);
20         dyn_arr2.sort;
21         $display("dyn_arr2 = %p", dyn_arr2);
22         dyn_arr2.rsort;
23         $display("dyn_arr2 = %p", dyn_arr2);
24         dyn_arr2.shuffle;
25         $display("dyn_arr2 = %p", dyn_arr2);
26
27     end
28
29 endmodule
```

```
1  module test_dynamic_array();
2
3      int dyn_arr2[] = '{9, 1, 8, 3, 4, 4}';
4      int dyn_arr1[];
5
6      initial begin
7
8          dyn_arr1 = new[6];
9
10         foreach (dyn_arr1[i]) begin
11             dyn_arr1[i] = i;
12         end
13
14         $display("dyn_arr1 = %p", "size = %0d", dyn_arr1, dyn_arr1.size());
15
16     end
17
18 endmodule
```

Question 2

1. Testbench code

```
1  module question_2_tb;
2
3      import question_2_pkg::*;    // Import package
4
5      // DUT signals
6      logic clk, rst_n, load_n, up_down, ce;
7      logic [WIDTH-1:0] data_load, count_out;
8      logic max_count, zero;
9
10     // Expected outputs
11     logic [WIDTH-1:0] count_out_expected;
12     bit expected_zero;
13     bit expected_max;
14
15     // Create object from class
16     question_2_class question_2_object;
17
18     // DUT instantiation (WIDTH overridden by package)
19     counter #(.WIDTH(WIDTH)) dut (
20         .clk(clk),
21         .rst_n(rst_n),
22         .load_n(load_n),
23         .up_down(up_down),
24         .ce(ce),
25         .data_load(data_load),
26         .count_out(count_out),
27         .max_count(max_count),
28         .zero(zero)
29     );
30
31     // Clock
32     always begin
33         | | #5 clk = ~clk;
34     end
35
```

```

36 // =====
37 // Tasks
38 // =====
39
40 task assert_reset();
41     rst_n = 0;
42     @(negedge clk);
43     rst_n = 1;
44 endtask
45
46
47 task golden_model (
48     input logic [WIDTH-1:0] prev_count,
49     input bit load_n, ce, up_down,
50     input logic [WIDTH-1:0] data_load,
51     output logic [WIDTH-1:0] expected,
52     output bit expected_max,
53     output bit expected_zero
54 );
55     if (!load_n)
56         expected = data_load;
57     else if (ce)
58         expected = up_down ? prev_count + 1 : prev_count - 1;
59     else
60         expected = prev_count;
61
62     expected_max = (expected == {WIDTH{1'b1}});
63     expected_zero = (expected == 0);
64 endtask
65

```

```

67 task check_result(
68     input logic [WIDTH-1:0] expected,
69     input bit expected_max,
70     input bit expected_zero
71 );
72     @(negedge clk);
73     if (count_out != expected)
74         $display("count_out mismatch: got %0d, expected %0d", count_out, expected);
75     if (max_count != expected_max)
76         $display("max_count mismatch: got %b, expected %b", max_count, expected_max);
77     if (zero != expected_zero)
78         $display("zero mismatch: got %b, expected %b", zero, expected_zero);
79     else
80         $display("test passed: count_out=%0d, max_count=%b, zero=%b", count_out, max_count, zero);
81 endtask
82

```

```

88   initial begin
89       clk = 0;
90       rst_n    = 1'b1;
91       load_n   = 1'b1;
92       ce       = 1'b0;
93       up_down  = 1'b1;
94       data_load = '0;
95
96       count_out_expected = '0;
97       expected_max       = 1'b0;
98       expected_zero      = 1'b1;
99
100      question_2_object = new();
101
102      // 1. Apply reset
103      assert_reset();
104
105      // 2. Run random tests
106      repeat (20) begin
107          assert(question_2_object.randomize());
108
109          load_n    = question_2_object.load_n_class;
110          ce        = question_2_object.ce_class;
111          assert_reset();
112          up_down   = question_2_object.up_down_class;
113          data_load = question_2_object.data_load_class;
114
115          golden_model(count_out, load_n, ce, up_down, data_load,
116                      | count_out_expected, expected_max, expected_zero);
117
118          check_result(count_out_expected, expected_max, expected_zero);
119      end
120      $finish;
121   end
122 endmodule

```

2. Package code

```
1 package question_2_pkg ;
2
3 parameter WIDTH = 4 ;
4
5 class question_2_class ;
6     rand logic rst_n_class, load_n_class, up_down_class, ce_class ;
7     rand logic [WIDTH-1:0] data_load_class ;
8
9     constraint distribution {
10         rst_n_class dist {1 := 80, 0 := 20}; // reset inactive most of the time
11         load_n_class dist {0 := 70, 1 := 30}; // load active 70%
12         ce_class dist {1 := 70, 0 := 30}; // enable active 70%
13     }
14 endclass
15
16 endpackage
```

3. Design code : 4.no bugs

```
8 module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
9 parameter WIDTH = 4;
10 input clk;
11 input rst_n;
12 input load_n;
13 input up_down;
14 input ce;
15 input [WIDTH-1:0] data_load;
16 output reg [WIDTH-1:0] count_out;
17 output max_count;
18 output zero;
19
20 always @(posedge clk) begin
21     if (!rst_n)
22         count_out <= 0;
23     else if (!load_n)
24         count_out <= data_load;
25     else if (ce)
26         if (up_down)
27             count_out <= count_out + 1;
28         else
29             count_out <= count_out - 1;
30 end
31
32 assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
33 assign zero = (count_out == 0)? 1:0;
34
35 endmodule
```

5. verification plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_1	When reset (rst_n=0) is asserted, the counter output count_out should reset to 0.	Directed at the start of simulation.	Coverpoint on rst_n assertion and count_out value = 0.	Checker compares DUT output count_out with expected = 0.
COUNTER_2	When load_n=0 , count_out should take the value of data_load.	Randomization of data_load values.	Coverpoint on load_n=0 with different data_load values across WIDTH.	Checker compares count_out with data_load.
COUNTER_3	When ce=1 and up_down=1 , counter increments by 1.	Randomization with ce=1, up_down=1.	Coverpoints on increment sequences; cross coverage of (ce, up_down).	Checker compares DUT count_out with golden model increment.
COUNTER_4	When ce=1 and up_down=0 , counter decrements by 1.	Randomization with ce=1, up_down=0.	Coverpoints on decrement sequences; cross coverage of (ce, up_down).	Checker compares DUT count_out with golden model decrement.
COUNTER_5	When ce=0 , counter should hold its previous value regardless of up_down.	Randomization with ce=0.	Coverpoint on stable values when ce=0.	Checker compares count_out with previous value.
COUNTER_6	When count_out reaches maximum ({WIDTH{1'b1}}), max_count should assert.	Directed + random sequences reaching maximum.	Coverpoint on hitting max value; bin on count_out ==	Checker compares DUT max_count with expected flag from golden model.
COUNTER_7	When count_out reaches 0, zero output should assert.	Directed + random sequences reaching 0.	Coverpoint on hitting zero value; bin on count_out == 0	Checker compares DUT zero with expected flag from golden model.
COUNTER_8	Check that counter works correctly for all valid WIDTH values (4, 6, 8).	Regression with parameter override.	Cross coverage of WIDTH values with corner cases (reset, load, up, down).	Checker compares outputs for each WIDTH configuration against golden model.

6. Do file

```

1  vlib work
2  vlog counter.v package.svh question_2_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.question_2_tb -cover
4  add wave *
5  coverage save question_2_tb.ucdb -onexit -du work.counter
6  run -all

```

7. Code Coverage report snippets

Coverage Report by instance with details

```
=====
=== Instance: /\question_2_tb#dut
=== Design Unit: work.counter
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	10	10	0	100.00%

```
=====Branch Details=====
```

Branch Coverage for instance /\question_2_tb#dut

Line	Item	Count	Source
----	----	----	-----
File counter.v			
-----IF Branch-----			
21		41	Count coming in to IF
21	1	21	if (!rst_n)
23	1	13	else if (!load_n)
25	1	5	else if (ce)
		2	All False Count

Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----			
26		5	Count coming in to IF
26	1	4	if (up_down)
28	1	1	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
32		34	Count coming in to IF
32	1	1	assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
32	2	33	assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
33                               34    Count coming in to IF
33              1              17    assign zero = (count_out == 0)? 1:0;

33              2              17    assign zero = (count_out == 0)? 1:0;
```

Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
-----	----	----	-----	-----
Conditions	2	2	0	100.00%

=====Condition Details=====

Condition Coverage for instance /\question_2_tb#dut --

File counter.v

-----Focused Condition View-----

Line 32 Item 1 (count_out == {4{{1}}})

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
(count_out == {4{{1}}})	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	(count_out == {4{{1}}})_0	-
Row 2:	1	(count_out == {4{{1}}})_1	-

-----Focused Condition View-----

Line 33 Item 1 (count_out == 0)

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
(count_out == 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count_out == 0)_0	-
Row 2:	1	(count_out == 0)_1	-

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	7	7	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\question_2_tb#dut --

Line	Item	Count	Source
8	File counter.v		module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
9			parameter WIDTH = 4;
10			input clk;
11			input rst_n;
12			input load_n;
13			input up_down;
14			input ce;
15			input [WIDTH-1:0] data_load;
16			output reg [WIDTH-1:0] count_out;
17			output max_count;
18			output zero;
19			

```

20          1          41    always @(posedge clk) begin
21                                if (!rst_n)
22          1          21        count_out <= 0;
23                                else if (!load_n)
24          1          13        count_out <= data_load;
25                                else if (ce)
26                                if (up_down)
27          1          4        count_out <= count_out + 1;
28                                else
29          1          1        count_out <= count_out - 1;
30                                end
31
32          1          35    assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
33          1          35    assign zero = (count_out == 0)? 1:0;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\question_2_tb#dut --

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\question_2_tb#dut --

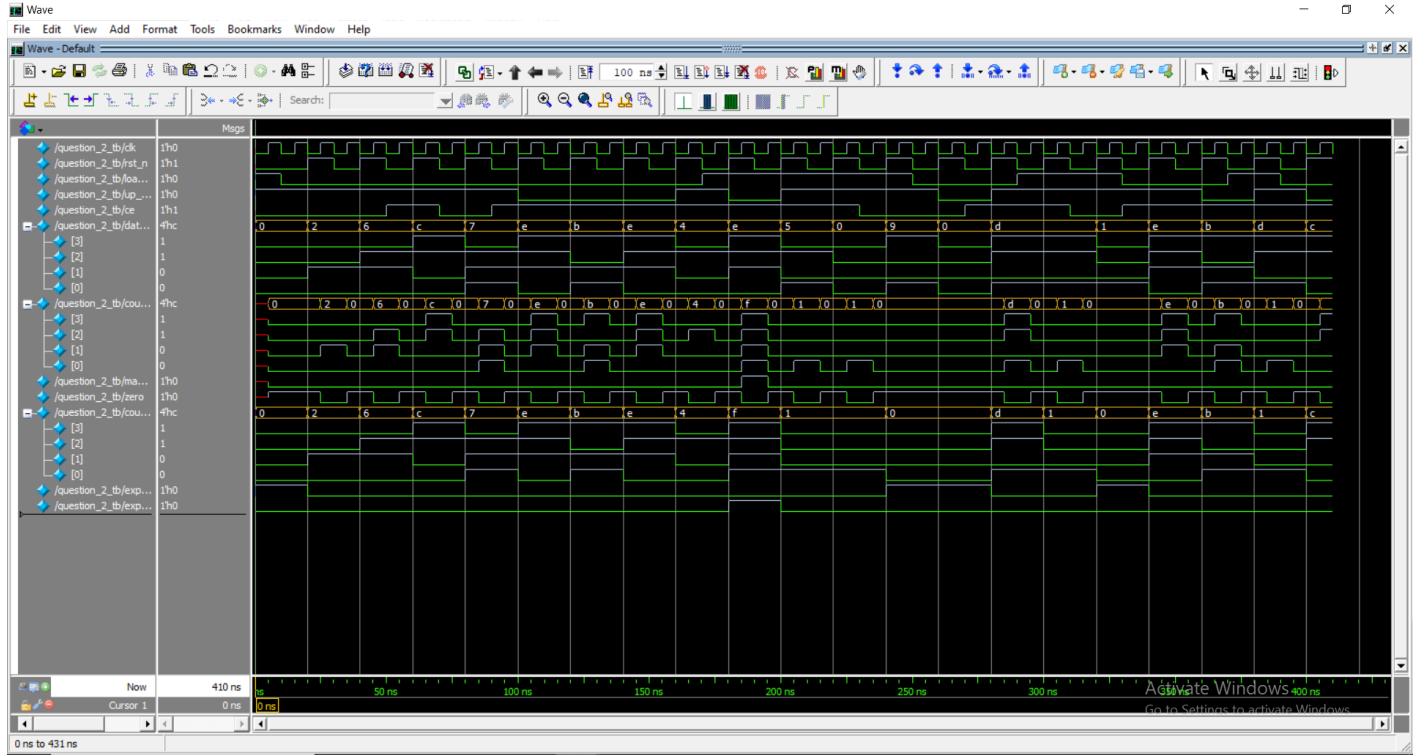
Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
ce	1	1	100.00
clk	1	1	100.00
count_out[3-0]	1	1	100.00
data_load[0-3]	1	1	100.00
load_n	1	1	100.00
max_count	1	1	100.00
rst_n	1	1	100.00
up_down	1	1	100.00
zero	1	1	100.00

Total Node Count = 15
Toggled Node Count = 15
Untoggled Node Count = 0

Toggle Coverage = 100.00% (30 of 30 bins)

Total Coverage By Instance (filtered view): 100.00%

8. QuestaSim Waveform



Question 3

1. Testbench code

```
1  module question_3_tb();
2
3      import alsu_pkg::*;
4
5      // =====
6      // DUT signals
7      // =====
8      logic clk, cin, serial_in, red_op_A, red_op_B, bypass_A, bypass_B, direction, rst;
9      opcode_e opcode;
10     logic signed [2:0] A, B;
11     logic [15:0] leds;
12     logic signed [5:0] out;
13
14     // =====
15     // Registered copies of DUT inputs for Golden Model
16     // =====
17     logic signed [2:0] A_reg, B_reg;
18     opcode_e opcode_reg;
19     logic red_op_A_reg, red_op_B_reg;
20     logic bypass_A_reg, bypass_B_reg;
21     logic direction_reg, serial_in_reg;
22     logic cin_reg;
23
24     // =====
25     // DUT output registers to avoid multiple drivers
26     // =====
27     logic signed [5:0] out_dut;
28     logic [15:0] leds_dut;
29
30     // Assign to DUT through internal wires only
31     assign out = out_dut;
32     assign leds = leds_dut;
33
34     //
```

```
34 // =====
35 // Instantiate DUT
36 // =====
37 ALSU dut (
38     .clk(clk), .cin(cin), .serial_in(serial_in),
39     .red_op_A(red_op_A), .red_op_B(red_op_B),
40     .bypass_A(bypass_A), .bypass_B(bypass_B),
41     .direction(direction), .rst(rst),
42     .opcode(opcode), .A(A), .B(B),
43     .leds(leds_dut), .out(out_dut)
44 );
45
46 // =====
47 // Clock generation
48 // =====
49 always #5 clk = ~clk;
50
```

```

51 // =====
52 // Input Register Stage (to match DUT internal behavior)
53 // =====
54 always @(posedge clk or posedge rst) begin
55     if (rst) begin
56         A_reg <= 0;
57         B_reg <= 0;
58         opcode_reg <= OR_OP;
59         red_op_A_reg <= 0;
60         red_op_B_reg <= 0;
61         bypass_A_reg <= 0;
62         bypass_B_reg <= 0;
63         direction_reg <= 0;
64         serial_in_reg <= 0;
65         cin_reg <= 0;
66     end else begin
67         A_reg <= A;
68         B_reg <= B;
69         opcode_reg <= opcode;
70         red_op_A_reg <= red_op_A;
71         red_op_B_reg <= red_op_B;
72         bypass_A_reg <= bypass_A;
73         bypass_B_reg <= bypass_B;
74         direction_reg <= direction;
75         serial_in_reg <= serial_in;
76         cin_reg <= cin;
77     end
78 end
79
80 // =====
81 // Transaction object
82 // =====
83 alsu_txn t;
84

```



```

85 // =====
86 // Golden model task
87 // =====
88 task golden_model
89 (
90     input logic signed [2:0] A, B,
91     input opcode_e opcode,
92     input logic red_op_A, red_op_B,
93     input logic bypass_A, bypass_B,
94     input logic direction, serial_in,
95     input logic cin,
96     output logic signed [5:0] exp_out
97 );
98     localparam string INPUT_PRIORITY = "A";
99     localparam string FULL_ADDER = "ON";
100
101     begin
102         exp_out = 0;
103         if (bypass_A && bypass_B) begin
104             exp_out = (INPUT_PRIORITY == "A") ? A : B;
105         end
106         else if (bypass_A) begin
107             exp_out = A;
108         end
109         else if (bypass_B) begin
110             exp_out = B;
111         end
112         else begin
113             case (opcode)
114             OR_OP: begin
115                 if (red_op_A && red_op_B)
116                     exp_out = (INPUT_PRIORITY == "A") ? (|A) : (|B);
117                 else if (red_op_A) exp_out = |A;
118                 else if (red_op_B) exp_out = |B;

```

```

116         exp_out = (INPUT_PRIORITY == "A") ? (|A) : (|B);
117     else if (red_op_A) exp_out = |A;
118     else if (red_op_B) exp_out = |B;
119     else exp_out = A | B;
120 end
121
122 XOR_OP: begin
123     if (red_op_A && red_op_B)
124         exp_out = (INPUT_PRIORITY == "A") ? (^A) : (^B);
125     else if (red_op_A) exp_out = ^A;
126     else if (red_op_B) exp_out = ^B;
127     else exp_out = A ^ B;
128 end
129
130 ADD_OP: begin
131     if (FULL_ADDER == "ON")
132         exp_out = A + B + cin;
133     else
134         exp_out = A + B;
135 end
136
137 MUL_OP: exp_out = A * B;
138
139     default: exp_out = 0; // Shift/Rotate not implemented yet
140 endcase
141 end
142 end
143 endtask
144
145 //

```

```
145 // =====
146 // Self-check initial block
147 // =====
148 initial begin
149     clk = 0;
150     cin      = 1'b0;
151     red_op_A = 1'b0;
152     red_op_B = 1'b0;
153     bypass_A = 1'b0;
154     bypass_B = 1'b0;
155     direction = 1'b0;
156     serial_in = 1'b0;
157     opcode    = ADD_OP;
158     A         = '0;
159     B         = '0;
160     rst       = 1;
161
162     // Hold reset for a few cycles
163     #12 rst = 0;
164
165     t = new();
166
```

```

167     repeat (1000) begin
168         logic signed [5:0] exp_out;
169         assert(t.randomize());
170
171         // Apply randomized inputs
172         cin      = t.cin;
173         serial_in = t.serial_in;
174         red_op_A  = t.red_op_A;
175         red_op_B  = t.red_op_B;
176         rst       = t.rst;
177         bypass_A  = t.bypass_A;
178         bypass_B  = t.bypass_B;
179         direction = t.direction;
180         opcode    = t.opcode;
181         A         = t.A;
182         B         = t.B;
183
184         // Wait a clock cycle to capture registered values
185         @(posedge clk);
186
187
188         // Golden model expected output
189         golden_model (
190             A_reg, B_reg, opcode_reg,
191             red_op_A_reg, red_op_B_reg,
192             bypass_A_reg, bypass_B_reg,
193             direction_reg, serial_in_reg,
194             cin_reg,
195             exp_out
196         );
197
198         // Compare results
199         if (out_dut !== exp_out) begin
200             $display("Mismatch at time %0t: opcode=%0d A=%0d B=%0d cin=%0d out=%0d exp=%0d",
201                 $time, opcode_reg, A_reg, B_reg, cin_reg, out_dut, exp_out);
202         end
203     end
204
205     $stop;
206 end
207
208 endmodule

```

2. Package code

```
1  package alsu_pkg;
2
3  // Enum for opcodes
4  typedef enum logic [2:0]
5  {OR_OP      = 3'h0, XOR_OP      = 3'h1,
6   ADD_OP      = 3'h2, MUL_OP      = 3'h3,
7   SHIFT_OP    = 3'h4, ROT_OP      = 3'h5,
8   INVALID_6   = 3'h6, INVALID_7   = 3'h7} opcode_e;
9
10 // Local parameters for edge cases for adder
11 localparam MAXPOS = 3;
12 localparam MAXNEG = -4;
13 localparam ZERO    = 0;
14
15 class alsu_txn;
16
17 // Inputs to be randomized
18 rand logic clk,rst,cin,serial_in,red_op_A,red_op_B;
19 rand logic bypass_A,bypass_B,direction;
20 rand opcode_e opcode;
21 rand logic signed [2:0] A, B;
22
23 // Constraints
24
25 // [C1: Reset low probability]
26 constraint c_reset {
27   rst dist { 1 := 1, 0 := 9 };
28 }
29
30 // [C2: A,B biased to MAXPOS, ZERO, MAXNEG for ADD/MUL]
31 constraint c_adder_bias {
32   if (opcode inside {ADD_OP, MUL_OP}) {
33     A dist { MAXPOS := 4, ZERO := 4, MAXNEG := 4, [-2:2] := 1 };
34     B dist { MAXPOS := 4, ZERO := 4, MAXNEG := 4, [-2:2] := 1 };
35   }
36 }
```

```

37
38 // [C3: OR/XOR + red_op_A high to A one-hot, B=0]
39 // [C4: OR/XOR + red_op_B high to B one-hot, A=0]
40 constraint c_red {
41     if (opcode inside {OR_OP, XOR_OP}) {
42         // red_op_A
43         if (red_op_A && !red_op_B) {
44             A inside {3'b001, 3'b010, 3'b100};
45             B == 3'b000;
46         }
47         //red_op_B
48         else if (red_op_B && !red_op_A) {
49             B inside {3'b001, 3'b010, 3'b100};
50             A == 3'b000;
51         }
52         //red_op_A & red_op_B
53         else if (red_op_A && red_op_B) {
54             A inside {3'b001, 3'b010, 3'b100};
55             B inside {3'b001, 3'b010, 3'b100};
56         }
57     }
58 }
59
60 // [C5: Invalid cases less frequent]
61 constraint c_invalid {
62     opcode dist {INVALID_6 := 1, INVALID_7 := 1, [OR_OP:ROT_OP] := 10};
63 }
64
65 // [C6: bypass disabled most of the time]
66 constraint c_bypass {
67     bypass_A dist {0 := 8, 1 := 2};
68     bypass_B dist {0 := 8, 1 := 2};
69 }
70

```

```

71      // [C7: No constraints on A,B if SHIFT/ROT]
72      // Handled implicitly by not applying extra restrictions
73
74      // [C8: cin relevant only for ADD_OP]
75      constraint c_cin_valid {
76          if (opcode == ADD_OP) {
77              cin dist {0 := 1, 1 := 1}; // both 0 and 1 equally likely
78          } else {
79              cin == 0; // cin is not used in other operations
80          }
81      }
82
83
84      endclass
85
86      endpackage
87

```

3. Design code

```

1  module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
2  parameter INPUT_PRIORITY = "A";
3  parameter FULL_ADDER = "ON";
4  input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5  input [2:0] opcode;
6  input signed [2:0] A, B;
7  output reg [15:0] leds;
8  output reg signed [5:0] out;
9
10 reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11 reg signed [1:0] cin_reg;
12 reg [2:0] opcode_reg;
13 reg signed [2:0] A_reg, B_reg;
14
15 wire invalid_red_op, invalid_opcode, invalid;
16
17 //Invalid handling
18 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
19 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20 assign invalid = invalid_red_op | invalid_opcode;
21
22 //Registering input signals
23 always @(posedge clk or posedge rst) begin
24     if(rst) begin
25         cin_reg <= 0;
26         red_op_B_reg <= 0;
27         red_op_A_reg <= 0;
28         bypass_B_reg <= 0;
29         bypass_A_reg <= 0;
30         direction_reg <= 0;
31         serial_in_reg <= 0;
32         opcode_reg <= 0;
33         A_reg <= 0;
34         B_reg <= 0;
35     end else begin
36         cin_reg <= cin;

```



```

37     red_op_B_reg <= red_op_B;
38     red_op_A_reg <= red_op_A;
39     bypass_B_reg <= bypass_B;
40     bypass_A_reg <= bypass_A;
41     direction_reg <= direction;
42     serial_in_reg <= serial_in;
43     opcode_reg <= opcode;
44     A_reg <= A;
45     B_reg <= B;
46 end
47 end
48
49 //leds output blinking
50 always @(posedge clk or posedge rst) begin
51     if(rst) begin
52         leds <= 0;
53     end else begin
54         if (invalid)
55             leds <= ~leds;
56         else
57             leds <= 0;
58     end
59 end
60
61 //ALSU output processing
62 always @(posedge clk or posedge rst) begin
63     if(rst) begin
64         out <= 0;
65     end
66     else begin
67         if (bypass_A_reg && bypass_B_reg)
68             out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
69         else if (bypass_A_reg)

```

```

69     else if (bypass_A_reg)
70         out <= A_reg;
71     else if (bypass_B_reg)
72         out <= B_reg;
73     else if (invalid)
74         out <= 0;
75     else begin
76         case (opcode_reg) //edited design error
77             3'h0: begin
78                 if (red_op_A_reg && red_op_B_reg)
79                     out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
80                 else if (red_op_A_reg)
81                     out <= |A_reg;
82                 else if (red_op_B_reg)
83                     out <= |B_reg;
84                 else
85                     out <= A_reg | B_reg;
86             end
87             3'h1: begin
88                 if (red_op_A_reg && red_op_B_reg)
89                     out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
90                 else if (red_op_A_reg)
91                     out <= ^A_reg;
92                 else if (red_op_B_reg)
93                     out <= ^B_reg;
94                 else
95                     out <= A_reg ^ B_reg;
96             end
97             3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error
98             3'h3: out <= A_reg * B_reg;
99             3'h4: begin
100                 if (direction_reg)
101                     out <= {out[4:0], serial_in_reg};

```

```

100         if (direction_reg)
101             out <= {out[4:0], serial_in_reg};
102         else
103             out <= {serial_in_reg, out[5:1]};
104     end
105     3'h5: begin
106         if (direction_reg)
107             out <= {out[4:0], out[5]};
108         else
109             out <= {out[0], out[5:1]};
110     end
111     default: out <= 0; //edit for 100% coverage and exclude it
112
113 endcase
114 end
115 end
116 end
117
118 endmodule

```

4. opcode_reg instead of opcode, add cin_reg for full adder and add default to exclude it

5. verification plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_1	When reset (rst) is asserted, output out and ledr should be cleared to 0.	Directed at the start of simulation (assert rst then deassert).	Cover reset asserted/deasserted.	Checker compares DUT output with expected - 0 when rst=1.
ALSU_2	When both bypass_A and bypass_B are enabled, output should follow priority input (INPUT_PRIORITY).	Randomize bypass_A, bypass_B, and A/B values.	Cover all permutations of bypass_A and bypass_B.	Checker validates correct priority logic.
ALSU_3	When bypass_A only is enabled, output should equal A.	Randomize bypass_A with different A.	Cover bypass_A=1, bypass_B=0.	Compare DUT output with A.
ALSU_4	When bypass_B only is enabled, output should equal B.	Randomize bypass_B with different B.	Cover bypass_A=0, bypass_B=1.	Compare DUT output with B.
ALSU_5	OR operation: if neither reduction signals are set, output = A B.	Randomize opcode=OR_OP, red_ap_A=0, red_ap_B=0.	Cover OR_OP with no reduction signals active.	Compare DUT output with A B.
ALSU_6	OR operation: if red_ap_A=1, output = OR-reduction of A (^).	A^).	Randomize opcode=OR_OP, red_ap_A=1.	Cover OR_OP with reduction on A.
ALSU_7	OR operation: if red_ap_B=1, output = OR-reduction of B (^).	B^).	Randomize opcode=OR_OP, red_ap_B=1.	Cover OR_OP with reduction on B.
ALSU_8	XOR operation: if neither reduction signals are set, output = A ^ B.	Randomize opcode=XOR_OP, red_ap_A=0, red_ap_B=0.	Cover XOR_OP with no reduction signals active.	Compare DUT output with A ^ B.
ALSU_9	ADD operation: output = A + B + cin (if full adder enabled).	Randomize opcode=ADD_OP, vary cin.	Cover cin = 0 and cin = 1.	Compare DUT output with A + B + cin.
ALSU_10	MUL operation: output = A × B.	Randomize opcode=MUL_OP, vary A and B.	Cover full range of A and B values.	Checker validates out = A × B.
ALSU_11	Shift left when direction=0 using serial_in.	Randomize opcode=SHIFT_0 P, direction=0, serial_in.	Cover direction=0 with both serial_in=0 and 1.	Checker ensures bitshift left correctly.
ALSU_12	Shift right when direction=1 using serial_in.	Randomize opcode=SHIFT_0 P, direction=1, serial_in.	Cover direction=1 with both serial_in=0 and 1.	Checker ensures bitshift right correctly.
ALSU_13	Randomize test to ensure stability under all conditions.	Fully randomize all inputs.	Functional coverage for all opcode, reduction, bypass, direction, serial_in.	Golden model comparison logs any mismatches.

6. Do file

```
1  vlib work
2  vlog ALSU.v package_q_3.svh question_3_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.question_3_tb -cover
4  add wave *
5  coverage save question_3_tb.ucdb -onexit -du work.ALSU
6  run -all
```

7. Code Coverage report snippets

code_cvr_alsu - Notepad

File Edit Format View Help

Coverage Report by instance with details

```
=====
=== Instance: /\question_3_tb#dut
=== Design Unit: work.ALSU
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	31	31	0	100.00%

```
=====Branch Details=====
```

Branch Coverage for instance /\question_3_tb#dut

Line	Item	Count	Source
----	----	----	-----
File ALSU.v			
-----IF Branch-----			
24		1089	Count coming in to IF
24	1	187	if(rst) begin
35	1	902	end else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
51		1089	Count coming in to IF
51	1	187	if(rst) begin
53	1	902	end else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
54		902	Count coming in to IF
54	1	425	if (invalid)
56	1	477	else

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
63          1078  Count coming in to IF
63          176  if(rst) begin
66          902  else begin
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
67          902  Count coming in to IF
67          38  if (bypass_A_reg && bypass_B_reg)
69          127  else if (bypass_A_reg)
71          137  else if (bypass_B_reg)
73          260  else if (invalid)
75          340  else begin
```

Branch totals: 5 hits of 5 branches = 100.00%

```
-----CASE Branch-----
76          340  Count coming in to CASE
77          175  3'h0: begin
87          84   3'h1: begin
97          19   3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error
98          17   3'h3: out <= A_reg * B_reg;
99          22   3'h4: begin
105         23   3'h5: begin
```

Branch totals: 6 hits of 6 branches = 100.00%

Activate Wind

code_cvr_alsu - Notepad

File Edit Format View Help

Branch totals: 6 hits of 6 branches = 100.00%

```
-----IF Branch-----
 78                               175    Count coming in to IF
 78             1                  18      if (red_op_A_reg && red_op_B_reg)

 80             1                   7      else if (red_op_A_reg)

 82             1                   2      else if (red_op_B_reg)

 84             1                  148      else
```

Branch totals: 4 hits of 4 branches = 100.00%

```
-----IF Branch-----
 88                               84    Count coming in to IF
 88             1                  18      if (red_op_A_reg && red_op_B_reg)

 90             1                  11      else if (red_op_A_reg)

 92             1                   4      else if (red_op_B_reg)

 94             1                  51      else
```

Branch totals: 4 hits of 4 branches = 100.00%

```
-----IF Branch-----
100                               22    Count coming in to IF
100             1                  14      if (direction_reg)

102             1                   8      else
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
106                               23    Count coming in to IF
106             1                  12      if (direction_reg)

108             1                   11      else
```

Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
-----	----	----	-----	-----
Conditions	6	6	0	100.00%

=====Condition Details=====

Condition Coverage for instance /\question_3_tb#dut --

File ALSU.v

-----Focused Condition View-----

Line 67 Item 1 (bypass_A_reg && bypass_B_reg)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
bypass_A_reg	Y		
bypass_B_reg	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	bypass_A_reg_0	-
Row 2:	1	bypass_A_reg_1	bypass_B_reg
Row 3:	1	bypass_B_reg_0	bypass_A_reg
Row 4:	1	bypass_B_reg_1	bypass_A_reg

-----Focused Condition View-----

Line 78 Item 1 (red_op_A_reg && red_op_B_reg)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
red_op_A_reg	Y		
red_op_B_reg	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	red_op_A_reg_0	-
Row 2:	1	red_op_A_reg_1	red_op_B_reg
Row 3:	1	red_op_B_reg_0	red_op_A_reg
Row 4:	1	red_op_B_reg_1	red_op_A_reg

-----Focused Condition View-----
Line 88 Item 1 (red_op_A_reg && red_op_B_reg)
Condition totals: 2 of 2 input terms covered = 100.00%

Input Term		Covered	Reason for no coverage	Hint
red_op_A_reg		Y		
red_op_B_reg		Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	-
Row 2:	1	red_op_A_reg_1	red_op_B_reg
Row 3:	1	red_op_B_reg_0	red_op_A_reg
Row 4:	1	red_op_B_reg_1	red_op_A_reg

Expression Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
Expressions	8	8	0	100.00%

=====Expression Details=====

Expression Coverage for instance /\question_3_tb#dut --

File ALSU.v

-----Focused Expression View-----
Line 18 Item 1 ((red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]))
Expression totals: 4 of 4 input terms covered = 100.00%

Input Term		Covered	Reason for no coverage	Hint
red_op_A_reg		Y		
red_op_B_reg		Y		
opcode_reg[1]		Y		
opcode_reg[2]		Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	((opcode_reg[1] opcode_reg[2]) && ~red_op_B_reg)
Row 2:	1	red_op_A_reg_1	((opcode_reg[1] opcode_reg[2]) && ~red_op_B_reg)

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	((opcode_reg[1] opcode_reg[2]) && ~red_op_B_reg)
Row 2:	1	red_op_A_reg_1	((opcode_reg[1] opcode_reg[2]) && ~red_op_B_reg)
Row 3:	1	red_op_B_reg_0	((opcode_reg[1] opcode_reg[2]) && ~red_op_A_reg)
Row 4:	1	red_op_B_reg_1	((opcode_reg[1] opcode_reg[2]) && ~red_op_A_reg)
Row 5:	1	opcode_reg[1]_0	((red_op_A_reg red_op_B_reg) && ~opcode_reg[2])
Row 6:	1	opcode_reg[1]_1	((red_op_A_reg red_op_B_reg) && ~opcode_reg[2])
Row 7:	1	opcode_reg[2]_0	((red_op_A_reg red_op_B_reg) && ~opcode_reg[1])
Row 8:	1	opcode_reg[2]_1	((red_op_A_reg red_op_B_reg) && ~opcode_reg[1])

-----Focused Expression View-----

Line 19 Item 1 (opcode_reg[1] & opcode_reg[2])

Expression totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
opcode_reg[1]	Y		
opcode_reg[2]	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	opcode_reg[1]_0	opcode_reg[2]
Row 2:	1	opcode_reg[1]_1	opcode_reg[2]
Row 3:	1	opcode_reg[2]_0	opcode_reg[1]
Row 4:	1	opcode_reg[2]_1	opcode_reg[1]

-----Focused Expression View-----

Line 20 Item 1 (invalid_red_op | invalid_opcode)

Expression totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
invalid_red_op	Y		
invalid_opcode	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	invalid_red_op_0	~invalid_opcode
Row 2:	1	invalid_red_op_1	~invalid_opcode
Row 3:	1	invalid_opcode_0	~invalid_red_op
Row 4:	1	invalid_opcode_1	~invalid_red_op

Statement Coverage:
Enabled Coverage
Statements 48 48 0 100.00%

=====Statement Details=====

Statement Coverage for instance /\question_3_tb#dut --

Line	Item	Count	Source
1			module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
2			parameter INPUT_PRIORITY = "A";
3			parameter FULL_ADDER = "ON";
4			input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5			input [2:0] opcode;
6			input signed [2:0] A, B;
7			output reg [15:0] leds;
8			output reg signed [5:0] out;
9			
10			reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11			reg signed [1:0] cin_reg;
12			reg [2:0] opcode_reg;
13			reg signed [2:0] A_reg, B_reg;

14			
15			wire invalid_red_op, invalid_opcode, invalid;
16			
17			//Invalid handling
18	1	855	assign invalid_red_op = (red_op_A_reg red_op_B_reg) & (opcode_reg[1] opcode_reg[2]);
19	1	768	assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20	1	497	assign invalid = invalid_red_op invalid_opcode;
21			
22			//Registering input signals
23	1	1089	always @(posedge clk or posedge rst) begin
24			if(rst) begin
25	1	187	cin_reg <= 0;
26	1	187	red_op_B_reg <= 0;
27	1	187	red_op_A_reg <= 0;
28	1	187	bypass_B_reg <= 0;
29	1	187	bypass_A_reg <= 0;
30	1	187	direction_reg <= 0;
31	1	187	serial_in_reg <= 0;
32	1	187	opcode_reg <= 0;
33	1	187	A_reg <= 0;
34	1	187	B_reg <= 0;



35			end else begin
36	1	902	cin_reg <= cin;
37	1	902	red_op_B_reg <= red_op_B;
38	1	902	red_op_A_reg <= red_op_A;
39	1	902	bypass_B_reg <= bypass_B;
40	1	902	bypass_A_reg <= bypass_A;
41	1	902	direction_reg <= direction;
42	1	902	serial_in_reg <= serial_in;
43	1	902	opcode_reg <= opcode;
44	1	902	A_reg <= A;
45	1	902	B_reg <= B;
46			end
47			end
48			
49			//leds output blinking
50	1	1089	always @(posedge clk or posedge rst) begin
51			if(rst) begin
52	1	187	leds <= 0;
53			end else begin
54			if (invalid)
55	1	425	leds <= ~leds;

```

56                                     else
57                                     1          477         leds <= 0;
58                                     end
59                                     end
60
61                                     //ALU output processing
62                                     1          1078        always @(posedge clk or posedge rst) begin
63                                     if(rst) begin
64                                     1          176         out <= 0;
65                                     end
66                                     else begin
67                                     if (bypass_A_reg && bypass_B_reg)
68                                     1          38         out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
69                                     else if (bypass_A_reg)
70                                     1          127        out <= A_reg;
71                                     else if (bypass_B_reg)
72                                     1          137        out <= B_reg;
73                                     else if (invalid)
74                                     1          260        out <= 0;
75                                     else begin
76                                     case (opcode_reg) //edited design error

```

```
77      3'h0: begin
78          if (red_op_A_reg && red_op_B_reg)
79              1      18      out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
80          else if (red_op_A_reg)
81              1      7      out <= |A_reg;
82          else if (red_op_B_reg)
83              1      2      out <= |B_reg;
84          else
85              1      148     out <= A_reg | B_reg;
86      end
87      3'h1: begin
88          if (red_op_A_reg && red_op_B_reg)
89              1      18      out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
90          else if (red_op_A_reg)
91              1      11      out <= ^A_reg;
92          else if (red_op_B_reg)
93              1      4      out <= ^B_reg;
94          else
95              1      51      out <= A_reg ^ B_reg;
96      end
97      1      19      3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error
```

```

98          1          17          3'h3: out <= A_reg * B_reg;
99
100          3'h4: begin
101          1          14          out <= {out[4:0], serial_in_reg};
102          else
103          1          8          out <= {serial_in_reg, out[5:1]};
104          end
105          3'h5: begin
106          if (direction_reg)
107          1          12          out <= {out[4:0], out[5]};
108          else
109          1          11          out <= {out[0], out[5:1]};

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	-----
Toggles	118	118	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\question_3_tb#dut --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
A[0-2]	1	1	100.00
A_reg[2-0]	1	1	100.00
B[0-2]	1	1	100.00
B_reg[2-0]	1	1	100.00
bypass_A	1	1	100.00
bypass_A_reg	1	1	100.00

Toggle Coverage for instance /\question_3_tb#dut --

Node	1H->0L	0L->1H	"Coverage"
A[0-2]	1	1	100.00
A_reg[2-0]	1	1	100.00
B[0-2]	1	1	100.00
B_reg[2-0]	1	1	100.00
bypass_A	1	1	100.00
bypass_A_reg	1	1	100.00
bypass_B	1	1	100.00
bypass_B_reg	1	1	100.00
cin	1	1	100.00
cin_reg[0]	1	1	100.00
clk	1	1	100.00
direction	1	1	100.00
direction_reg	1	1	100.00
invalid	1	1	100.00
invalid_opcode	1	1	100.00
invalid_red_op	1	1	100.00
leds[15-0]	1	1	100.00
opcode[0-2]	1	1	100.00
opcode_reg[2-0]	1	1	100.00
out[5-0]	1	1	100.00
red_op_A	1	1	100.00
red_op_A_reg	1	1	100.00
red_op_B	1	1	100.00
red_op_B_reg	1	1	100.00
rst	1	1	100.00
serial_in	1	1	100.00
serial_in_reg	1	1	100.00

Total Node Count	=	59	
Toggled Node Count	=	59	
Untoggled Node Count	=	0	
Toggle Coverage	=	100.00% (118 of 118 bins)	

Total Coverage By Instance (filtered view): 100.00%

8. QuestaSim waveform

